

Dokumentacja wdrożeniowa

1. Deployment – ogólny opis aplikacji
 - Cel i zakres dokumentacji
 - Architektura i technologia
 - Struktura rozwiązania i główne komponenty
 - Środowiska i hosting
2. DEV/TEST – Instalacja i uruchomienie lokalne
 - Wymagania wstępne
 - Konfiguracja aplikacji
 - Uruchomienie aplikacji lokalnie
 - Weryfikacja działania
3. Produkcja – Wdrożenie (Windows + IIS)
 - Wymagania infrastruktury
 - Przygotowanie IIS pod aplikację
 - Budowa i publikacja aplikacji
 - Konfiguracja aplikacji na serwerze produkcyjnym
4. Baza danych – produkcja
 - Tworzenie bazy danych
 - Tworzenie konta i nadawanie uprawnień
 - Zbudowanie connection string dla aplikacji
 - Uruchomienie migracji

1. Deployment – ogólny opis aplikacji

Cel i zakres dokumentacji

Dokumentacja opisuje proces budowania, konfiguracji i wdrażania aplikacji **DnDCC** w środowiskach **Developerskich**, **testowych** oraz **produkcyjnych**, z wykorzystaniem **Windows + IIS** i bazy danych **SQL Server**. Dokument pełni rolę punktu wejścia dla administratorów, deweloperów i osób odpowiedzialnych za utrzymanie.

Architektura i technologia

Aplikacja DnDCC jest systemem webowym opartym na platformie **.NET 8** i architekturze **ASP.NET Core MVC + Web API** z warstwą dostępu do danych zrealizowaną przy użyciu **Entity Framework Core** oraz bazy **SQL Server**.

Aplikacja wspiera uwierzytelnianie użytkowników (Identity/**JWT**), generowanie i obsługę kart postaci oraz korzysta z plików statycznych serwowanych z katalogu **wwwroot**.

Struktura rozwiązania i główne komponenty

Rozwiązanie składa się z kilku projektów logicznie wydzielających warstwy:

- **Projekt.Web** – warstwa prezentacji i API:
 - pliki konfiguracyjne: **appsettings.json** , **appsettings.Development.json**
 - punkt wejścia aplikacji: **Program.cs**
 - kontrolery MVC/API, widoki Razor (**Views**), zasoby statyczne (**wwwroot**)
- **Projekt.DAL** – warstwa dostępu do danych:
 - kontekst EF Core: **ApplicationDbContext.cs**
 - migracje bazy danych: katalog **Migrations/**
 - projekt: **Projekt.DAL.csproj**
- **Projekt.Model** – modele domenowe:
 - klasy danych, np. **Character** , **Item** , **User** , **Role** w katalogu **DataModels/**
- **Projekt.Services** – logika biznesowa:
 - serwisy aplikacyjne (**ConcreteServices**), konfiguracja (np. AutoMapper)
- **Projekt.ViewModel** – modele widoków:
 - klasy ViewModel (**VM/**) używane do przekazywania danych do warstwy prezentacji

Środowiska i hosting

Aplikacja może być wdrażana w dwóch głównych typach środowisk:

- **DEV/TEST** – środowiska deweloperskie i testowe, służące do bieżącego rozwoju, testów manualnych i integracyjnych.
- **PROD** – środowisko produkcyjne, przeznaczone dla użytkowników końcowych. Aplikacja uruchamiana jest na serwerach **Windows Server + IIS** z włączonym HTTPS, osobną instancją **SQL Server PROD** oraz dodatkowymi wymaganiami dotyczącymi bezpieczeństwa (sekrety, JWT, connection strings)

2. DEV/TEST - Instalacja i uruchomienie lokalne

Wymagania wstępne

Na lokalnym komputerze muszą być zainstalowane:

- **.NET SDK** w wersji 8.0
- **SQL Server** np:
 - **SQL Server Express**
 - **LocalDB**
- **Git**
- **Visual Studio** lub **Visual Studio Code**

Następnie wymagane jest sklonowanie repozytorium z githuba

```
1 git clone https://github.com/TheBigWhiteDuck/DnDCC.git
```

Utwórz pustą bazę danych (np. `DnDCC_Dev`) i przygotuj connection string do bazy, która będzie używana wyłącznie w DEV/TEST.

Przykładowy wygląd Connection String:

```
1 "ConnectionStrings": {  
2   "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=DnDCC_Dev;Trusted_Connection=True;MultipleActiveResultSets=true;  
3 }
```

Konfiguracja aplikacji

Konfiguracja środowiskowa znajduje się w `Projekt.Web\\appsettings.json`

Dla lokalnego środowiska aplikacji należy:

1. Ustawić **connection string**
2. Ustawić parametry **JWT**:
 - a. `Jwt:Issuer`
 - b. `Jwt:Audience`
 - c. `Jwt:Key`
 - d. `Jwt:ExpiryMinutes`
3. Parametry **Stripe**:
 - a. `SecretKey`
 - b. `WebhookSecret`
4. Konfiguracja **OpenAI:ApiKey** - Ustaw zmienną środowiskową w Windows poprzez PowerShell:

```
1 setx OpenAI__ApiKey "sk-...twoj_klucz_dev..."
```

4. Przywracanie pakietów

```
1 dotnet restore
```

5. Migracja bazy danych

```
1 dotnet ef database update --project .\\Projekt.DAL\\Projekt.DAL.csproj --startup-project .\\Projekt.Web\\
```

Uruchomienie aplikacji lokalnie

Aby uruchomić wcześniej przygotowaną aplikację należy przejść do katalogu głównego projektu oraz następnie uruchomić poleceniem:

```
1 dotnet run
```

i Aplikacja jest już uruchomiona pod adresem podanym w terminalu.

Weryfikacja działania

Po uruchomieniu:

1. Przejdź na adres, np. `https://localhost:5000`.
2. Sprawdź:
 - ładowanie strony startowej,
 - logowanie/rejestrację,
 - dodanie postaci
 - ładowanie zasobów statycznych (CSS, JS, grafiki)
3. Sprawdź PDF:
 - w katalogu `Projekt.Web\wwwroot\pdfs` powinien znajdować się wymagany plik (np. `Players_Handbook.pdf`),
 - Sprawdź poprawne otwieranie i przeglądanie PDF
4. Sprawdź generowanie awatara:
 - zaloguj się na konto z uprawnieniem premium (wymagane przez `AvatarController`),
 - wywołaj funkcję generowania

3. Produkcja - Wdrożenie (Windows + IIS)

Wymagania infrastruktury

1. **Windows Server**
2. **Internet Information Services (IIS)** z komponentami:
 - Web Server,
 - ASP.Net Core Hosting Bundle dla .NET 8
3. **SQL Server**
4. **Stripe**
5. Certyfikat **TLS** jeśli aplikacja ma być dostępna przez HTTPS

Przygotowanie IIS pod aplikację

1. Instalacja ASP.NET Core Hosting Bundle (.NET 8)
2. Application Pool:
 - utwórz nowy App Pool, np. `DnDCC_AppPool`,
 - wybierz .NET CLR version: „No Managed Code”,
 - następnie skonfiguruj Managed pipeline mode: „Integrated”,
3. Site / Virtual Directory:
 - utwórz nową stronę IIS, np. `DnDCC_Prod`,
 - Ścieżka dostępu: katalog na dysku, np. `D:\Sites\DnDCC\Prod`,
 - przypisz App Pool: `DnDCC_AppPool`,
 - dodaj powiązania:
 - `https` + port (np. 443) + host name (domena produkcyjna),
 - podłącz certyfikat TLS jeżeli używany.
4. Uprawnienia NTFS:
 - nadaj konto serwisowe App Pool do katalogu aplikacji:
 - uprawnienia jakie musi posiadać to co najmniej `Read & Execute`,
 - dodatkowo `Write` dla `wwwroot\images\avatars`

Budowa i publikacja aplikacji

Publikacja może zostać wykonana na serwerze CI/CD lub na stacji roboczej.

1. Na maszynie gdzie budowane jest projekt przejdź do katalogu rozwiązania.
2. Przywróć pakiety i zbuduj

```
1 dotnet restore
2 dotnet build .\Projekt.sln -c Release
```

3. Opublikuj projekt webowy w trybie Release do katalogu publikacji.

```
1 dotnet publish .\Projekt.Web\Projekt.csproj -c Release -o .\publish
```

4. Spakuj zawartość katalogu `.\publish` na przykład do ZIP jako paczka wdrożeniowa i przenieś ją na serwer Produkcyjny.

Konfiguracja aplikacji na serwerze produkcyjnym

Pierwszym korkiem jest rozpakowanie paczki z naszym kodem do katalogu ustalonego wcześniej np. `D:\Sites\DnDCC\Prod`

Następnie ustalamy zmienne środowiskowe systemowe:

- `ConnectionStrings__DefaultConnection` connection string do bazy produkcyjnej
- `Jwt__Issuer`, `Jwt__Audience`, `Jwt__Key`, `Jwt__ExpiryMinutes` klucz JWT w produkcji powinien być losowym, długim ciągiem znaków.
- `SecretKey`, `WebhookSecret` klucze dla konfiguracji płatności Stripe.
- `OpenAI__ApiKey` klucz API OpenAI używany do generowania awatarów przez `AvatarController`.

Przykładowy zestaw komend:

```
1 setx ConnectionStrings__DefaultConnection "Server=sql-prod;Database=DnDCC_Prod;User_Id=...;Passwo
2 setx Jwt__Key "losowy_bezpieczny_klucz_JWT"
3 setx OpenAI__ApiKey "sk-...prod..."
```

4. Baza danych - produkcja

Tworzenie bazy danych

1. Zaloguj się do **SQL Server** - SQL Server Management Studio (SSMS)
2. Tworzenie nowej bazy danych
 - a. Database name: `DnDCC_Prod`
 - b. Collation: `SQL_Latin1_General_CP1_CI_AS`
 - c. Recovery model: `FULL`
 - d. Initial Size: `10MB`

Tworzenie konta i nadawanie uprawnień

Do poprawnego i bezpiecznego działania bazy danych wymagane będzie stworzenie podanych w tabeli Użytkowników i przypisanie im odpowiednich uprawnień:

Nazwa konta	Uwierzytelnianie	Przeznaczenie	Zakres uprawnień w bazie <code>DnDCC_Prod</code>
<code>DnDCC_AppUser</code>	Login SQL	Konto aplikacyjne, używane w connection stringu przez aplikację webową DnDCC	<code>db_datareader</code> , <code>db_datawriter</code>
<code>DnDCC_DeployUser</code>	Login SQL	Konto techniczne do wykonywania migracji EF Core i zmian schematu	<code>db_owner</code>
<code>DBA_Admin</code>	Login SQL / Windows auth	Administracja SQL Server, konfiguracja instancji, backup, monitoring	<code>sysadmin</code>

Zbudowanie connection string dla aplikacji

Na podstawie powyższych danych tworzymy connection string, którego użyje aplikacja DnDCC.

Przykład z logowaniem poprzez Login SQL:

```
1 Server=SQLPR0D01;Database=DnDCC_Prod;User_Id=DnDCC_AppUser;Password=<haslo>;Encrypt=True;TrustServerCertificate=False;MultipleActiveResultSets=True
```

Parametry:

- **Server** - adres instancja SQL Server
- **Database** - nazwa bazy
- **User_Id / Password** - dane do logowania
- **Encrypt=True** - wymusza szyfrowanie połączenia
- **TrustServerCertificate=False** - false jeżeli skonfigurowany jest certyfikat serwera
- **MultipleActiveResultSets=True** - wymagane przez EF Core

• Następnie na serwerze z IIS należy ustawić nową zmienną środowiskową **ConnectionStrings__DefaultConnection**

```
1 setx ConnectionStrings__DefaultConnection "Server=...;" /M
```

Uruchomienie migracji

Aby stworzyć nową migrację produkcyjną trzeba w katalogu rozwiązania ./Projekt wykonać:

```
1 dotnet restore
2 dotnet ef database update --project .\Projekt.DAL\Projekt.DAL.csproj --startup-project .\Projekt.Web\Projekt.csproj
```

Po wykonaniu migracji aplikacja webowa korzysta z nowej produkcyjnej bazy danych.